~ Complete Step by Step ~

React JS

Components • Hooks • Examples • Diagrams

Index

1 Introduction to React
15 Performance Hooks
2 Setup & Project Structure
16 Custom Hooks
3 JSX
17 Rendering & Virtual DOM
4 Components
18 React Router
5 Props
19 Data Fetching
6 State (useState)
20 Styling
7 Events
21 State Management (Redux, Zustand)
8 Forms
22 Patterns & Rules of Hooks
9 Conditional Rendering
23 Advanced: Suspense, Portals, React 19
10 Lists & Keys
24 Testing & Deployment
11 useEffect & Lifecycle
25 Mini Project: Todo App
12 useRef & the DOM
26 Interview Q&A
13 useContext
27 Cheat Sheet
14 useReducer

How to use

- Needs **HTML, CSS and modern JavaScript** first (arrow functions, destructuring, spread, map/filter, modules, async/await).
- ***code*** in blue/green boxes = examples to **type yourself**. Draw the **diagrams** in your notebook.
- All examples use **function components + Hooks** (modern React 18/19).

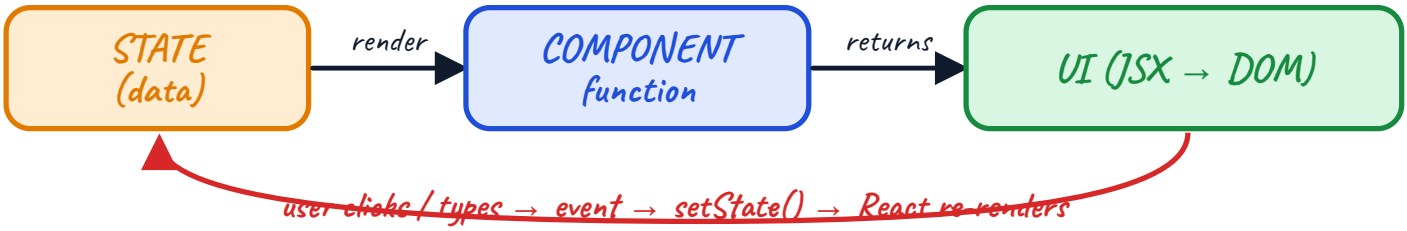
Notes by asadokz18 ✿

1. Introduction to React

Definition

React is a free, open-source **JavaScript library for building user interfaces** (UI), made by **Facebook (Meta)** in 2013. You describe *what the UI should look like* for a given data (**declarative**) and React updates the browser efficiently.

The core idea

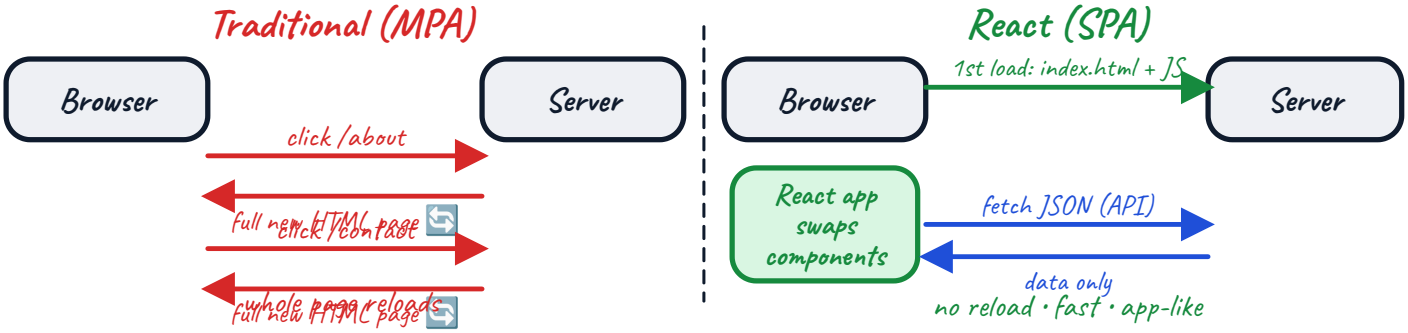


UI = f(state) — change the data, React updates the screen for you

Why React? (features)

- → **Component-based** – small reusable pieces (Button, Card, Navbar)
- → **Declarative** – you say *what*, not *how*
- → **Virtual DOM** – fast, minimal real-DOM updates
- → **One-way data flow** – predictable, easy to debug
- → **Huge ecosystem** – Router, Redux, Next.js, React Native
- → **JSX** – HTML-like syntax inside JavaScript
- → **Hooks** – state & lifecycle in functions
- → **Learn once, write anywhere** – web, mobile (React Native)

Single Page Application (SPA)



Vanilla JS vs React – example

Vanilla JS

```
let n = 0;
const el = document.getElementById("n");
btn.onclick = () => {
  n++;
  el.textContent = n;    // manual
};
```

React

```
function Counter() {
  const [n, setN] = useState(0);
  return (
    <button onClick={() => setN(n+1)}>
      Clicked {n}
    </button>);    // automatic
}
```

Who uses React?

Facebook, Instagram, Netflix, Airbnb, WhatsApp Web, Discord, Dropbox... It is the most popular front-end library.

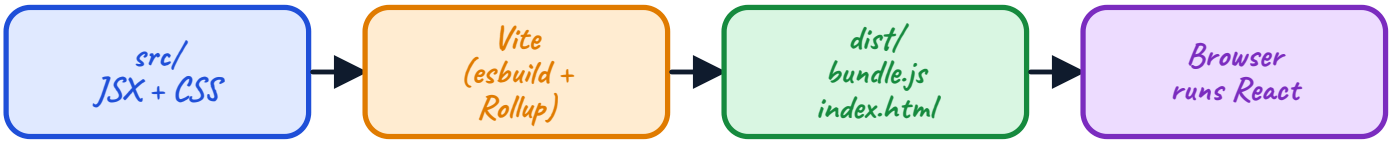
2. Setup & Project Structure

Requirements

→ **Node.js** (LTS) – check ***node -v*** and ***npm -v*** · a code editor (**VS Code**) · browser + **React Developer Tools** extension.

Create a React app with Vite (recommended)

```
npm create vite@latest my-app -- --template react      # (or react-ts for TypeScript)
cd my-app
npm install
npm run dev          # → http://localhost:5173  (hot reload)
```



npm run dev → instant dev server (HMR) / npm run build → optimised files for hosting

Folder structure

my-app/

```
my-app/
├── node_modules/   # packages
├── public/         # static files
├── src/
│   ├── assets/     # images
│   ├── components/ # reusable UI
│   ├── App.jsx     # root component
│   ├── App.css
│   ├── main.jsx    # entry point
│   └── index.css
├── index.html      # has <div id="root">
├── package.json    # scripts + deps
└── vite.config.js
```

File	Role
index.html	Single HTML page with ** **
main.jsx	Starts React & mounts **<App/>**
App.jsx	Top component
package.json	name, scripts (dev/build/preview), dependencies

Scripts: ***npm run dev*** · ***build*** · ***preview*** · ***lint***

src/main.jsx

```jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

createRoot(document.getElementById("root")).render(    // mount point
  <StrictMode>          {/* extra dev checks – runs effects twice in dev */}
    <App />
  </StrictMode>
);
```

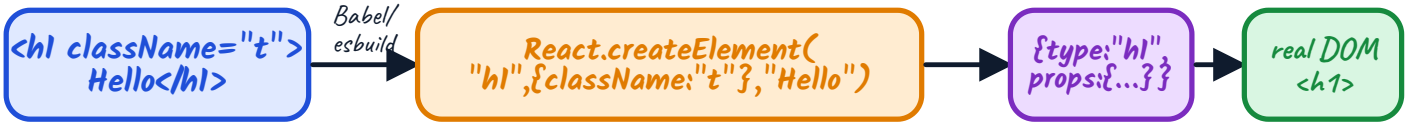
src/App.jsx – your first component

```jsx
function App() {
  return <h1>Hello React, I am asadokz18 👋</h1>;
}
export default App;
```

Tip: files use **.jsx**, components start with a **Capital**. Avoid deprecated **create-react-app**; use Vite or Next.js.

3. JSX – JavaScript XML

JSX lets you write **HTML-like markup inside JavaScript**. It is compiled to ***React.createElement()*** calls.



JSX is just syntax sugar – the browser never sees JSX

Rules of JSX (very important)

Rule	Example
1. Return **ONE parent** element (or a **Fragment**)	`<> <h1/> <p/> </>`
2. **Close every tag** (even void tags)	`<img /> <input />  `
3. **camelCase** attributes; **className**, **htmlFor**	`className="box" onClick tabIndex`
4. { } runs JavaScript **expressions**	`{name} {2+3} {fn()} {a ? x : y}`
5. **style** is an **object** with camelCase	`style={{ backgroundColor: "red" }}`
6. Comments inside JSX	`{/* comment */}`
7. Components start with a **Capital**	`<Card /> (lowercase = HTML tag)`

Example – expressions in JSX

```jsx
function Profile() {
  const user = { name: "Asad", age: 21, img: "/me.png" };
  const isOnline = true;
  return (
    <>                                          {/* Fragment – no extra div */}
      <h1 className="title">Hello, {user.name}!</h1>
      <img src={user.img} alt={user.name} width={100} />
      <p>Next year I will be {user.age + 1}</p>          {/* expression */}
      <p style={{ color: isOnline ? "green" : "gray", fontSize: "20px" }}>
        {isOnline ? "Online" : "Offline"}
      </p>
      <label htmlFor="n">Name</label> <input id="n" />
    </>
  );
}
```

Only expressions!

Inside { } you can put values, calls, ternaries – **not** ***if***, ***for*** statements or objects to print (`{ {a:1} }` → error).

Why the rules?

JSX becomes JS: ***class*** and ***for*** are JS keywords, so React uses ***className*** / ***htmlFor***.

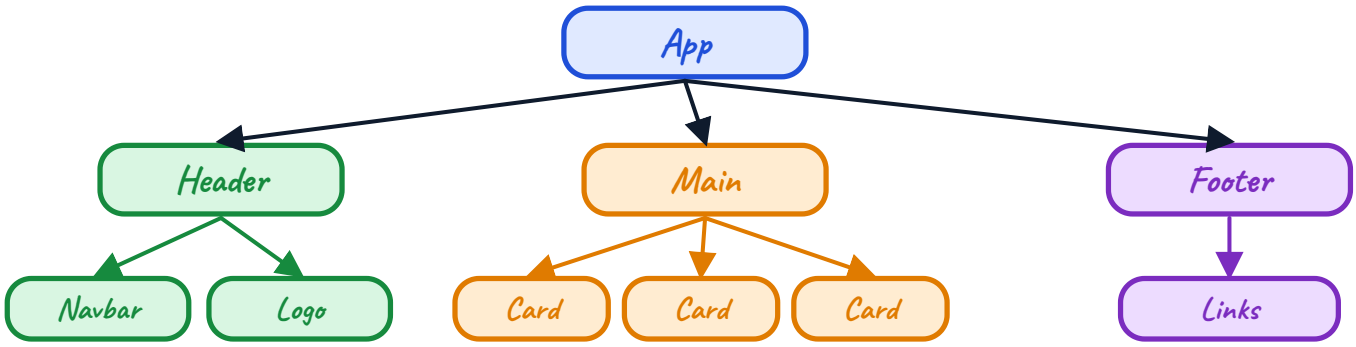
JSX also...

- **Prevents XSS**: values in { } are escaped automatically. Raw HTML only via ***dangerouslySetInnerHTML*** (avoid).
- Can be stored in variables / returned / passed: ***const el = <h1>Hi</h1>;***
- Multi-line JSX → wrap in ( ) after ***return***. Boolean attr: ***disabled*** = ***disabled={true}***.

4. Components

Component

A **reusable, independent piece of UI** – a JavaScript **function that returns JSX**. Name must start with a **Capital letter**. Components are like LEGO blocks.



React app = a TREE of components (parent → children)

components/Card.jsx

```jsx
function Card() {                  // 1. define (function + JSX)
  return (
    <div className="card">
      <h3>React Course</h3>
      <p>Learn components step by step.</p>
    </div>
  );
}
export default Card;               // 2. export (default)
```

App.jsx – import, nest and reuse

```jsx
import Card from "./components/Card";              // default import (any name)
import { Navbar, Footer } from "./components/Layout";   // named imports (exact names)

function App() {
  return (
    <div>
      <Navbar />
      <Card /> <Card /> <Card />            {/* reuse 3 times */}
      <Footer />
    </div>
  );
}
export default App;
```

Export type	Syntax
default (1 per file)	`export default Card;` → `import Card from "./Card";`
named (many)	`export function Navbar(){}` → `import { Navbar } from "./Layout";`

Good component habits

- One component = **one job** (single responsibility)
- **Never define a component inside another** (it is re-created every render, loses state)
- Keep them small; split big UI into children
- Components must be **pure**: same props/state → same JSX

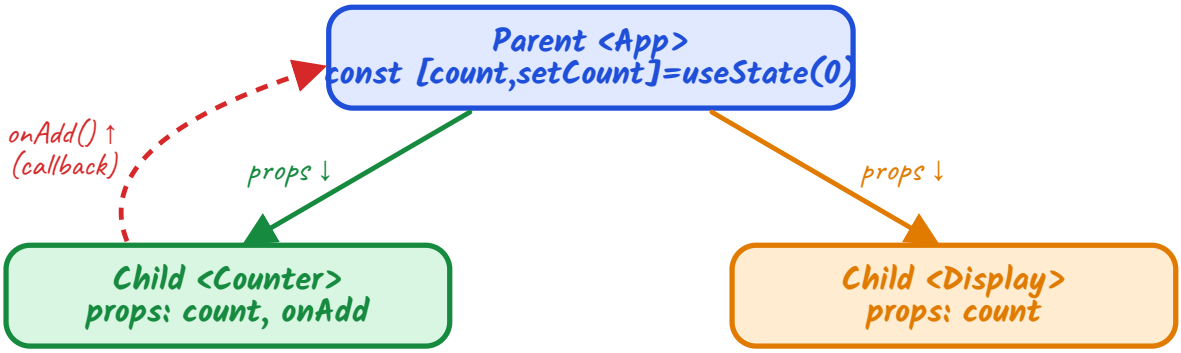
Class components (old way)

Class = ***this.state***, ***this.setState()***, ***componentDidMount/DidUpdate/WillUnmount***. Today use **function components + Hooks**.

5. Props (Properties)

Props

Inputs to a component, passed from parent to child like HTML attributes. Props are **read-only** – a component must never change its own props.



Data flows DOWN (props) • events flow UP (callbacks) = one-way data flow

Passing & receiving props

```
function Card({ title, price, inStock = true }) {      // destructure + default value
  return (
    <div className="card">
      <h3>{title}</h3> <p>Price: ${price}</p>
      <p>{inStock ? "Available" : "Sold out"}</p>
    </div>
  );
}
function App() {
  return (<>
    <Card title="Laptop" price={999} />                  {/* strings: "..."  others: {...} */}
    <Card title="Phone" price={599} inStock={false} />
  </>);
}
```

Feature	Example
props object (no destructuring)	`function Card(props){ return <h3>{props.title}</h3> }`
Pass any type	`<Box n={5} arr={[1,2]} obj={{a:1}} fn={handle} flag />`
Spread props	`const p = {"{title:'A', price:5}"}; <Card {...p} />`
children prop	`<Panel><p>Hi</p></Panel>` → `function Panel({children})`
Pass a **function** (callback)	`<Button onAdd={() => setN(n+1)} />`

children – wrapper components

```
function Panel({ title, children }) {
  return <section className="panel"><h2>{title}</h2>{children}</section>;
}
<Panel title="News"> <p>React 19 is out!</p> <button>Read</button> </Panel>
```

Never mutate props

`props.title = "x"` ❌ – props are immutable. To change data, use **state** in the parent.

Prop drilling

Passing props through many levels is tedious → use **Context** (Chapter 13) or a state library.

Validate props: TypeScript types (best) or **PropTypes** package: **`Card.propTypes = {"{ title: PropTypes.string.isRequired }"}`**.
Props vs State: props come **from outside** (read-only); state lives **inside** the component (changeable).

6. State & the useState Hook

State

Data that **changes over time** and that the component **remembers** between renders. When state changes, React **re-renders** the component. Normal variables (***let n = 0***) do **not** trigger updates and reset every render.



State is a SNAPSHOT per render · setState does not change the variable immediately – it asks React to render again

Counter – the classic example

```jsx
import { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);        // [current value, updater function] = initial value
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>+</button>
      <button onClick={() => setCount(count - 1)}>-</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

State rules & patterns

Situation	Correct code
Update based on **previous** value	`setCount(prev => prev + 1)` ✔ functional update
Object state (**don't mutate**)	`setUser({"{...user, age: 22}"})` ✘ `user.age = 22`
Array – **add**	`setItems([...items, newItem])`
Array – **remove**	`setItems(items.filter(i => i.id !== id))`
Array – **update** one	`setItems(items.map(i => i.id===id ? {"{...i, done:true}"} : i))`
Lazy initial value	`useState(() => JSON.parse(localStorage.getItem("x")))`
Several states	`const [name,setName]=useState(""); const [age,setAge]=useState(0);`

Updating object & array state immutably

```jsx
const [user, setUser] = useState({ name: "Asad", age: 21 });
setUser({ ...user, age: 22 });                  // copy old fields, change one
const [todos, setTodos] = useState(["HTML", "CSS"]);
setTodos([...todos, "React"]);                  // new array – React sees a new reference
```

Gotchas

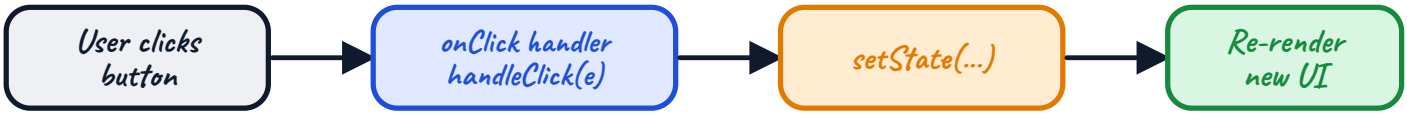
- **Never** change state directly (***count++***, ***arr.push()***).
- ***setCount(count+1)*** three times = +1 only (same snapshot). Use ***prev =>***.
- State updates are **batched** & async.

Lifting state up

If two siblings need the same data, move the state to their **closest common parent** and pass it down with props.

7. Handling Events

React events are written in **camelCase** and receive a **function**: ***onClick***, ***onChange***, ***onSubmit***, ***onMouseEnter***, ***onKeyDown***, ***onFocus***, ***onBlur***...



Pass the FUNCTION, do not call it: onClick={handle} ✔ onClick={handle()} ✘ (runs immediately)

Different ways to write handlers

```
function Demo() {
  const [n, setN] = useState(0);
  function handleClick() { setN(n + 1); }                  // 1. named function
  const say = (msg) => alert(msg);
  return (<>
    <button onClick={handleClick}>Add</button>              {/* pass reference */}
    <button onClick={() => setN(n - 1)}>Minus</button>      {/* inline arrow */}
    <button onClick={() => say("Hi")}>Say hi</button>       {/* with arguments → wrap in arrow */}
    <button onClick={(e) => console.log(e.target)}>Event</button>   {/* event object */}
  </>);
}
```

Event	When / example
onClick	mouse click on element
onChange	input / select / textarea value changes (every keystroke)
onSubmit	form submitted · use **e.preventDefault()** to stop page reload
onKeyDown / onKeyUp	**if (e.key === "Enter") ...**
onMouseEnter / Leave	hover in / out
onFocus / onBlur	input gets / loses focus

Event object & propagation

- **e** is a **SyntheticEvent** - a cross-browser wrapper of the native event (***e.target***, ***e.target.value***, ***e.key***, ***e.currentTarget***, ***e.nativeEvent***).
- ***e.preventDefault()*** - stop default behaviour (form submit, link navigation). ***e.stopPropagation()*** - stop **bubbling** to parent handlers.
- Events **bubble** up: clicking a child also fires the parent's ***onClick***. Capture phase: ***onClickCapture***.

Passing handlers as props (child → parent communication)

```
function Child({ onAdd }) { return <button onClick={() => onAdd(5)}>+5</button>; }
function Parent() {
  const [total, setTotal] = useState(0);
  return <><h3>{total}</h3><Child onAdd={(x) => setTotal(total + x)} /></>;   // naming: onXxx for props, handleXxx for functions
}
```

Common bug

onClick={handle()} calls the function during render → infinite loop. Write ***onClick={handle}*** or ***{() => handle(5)}***.

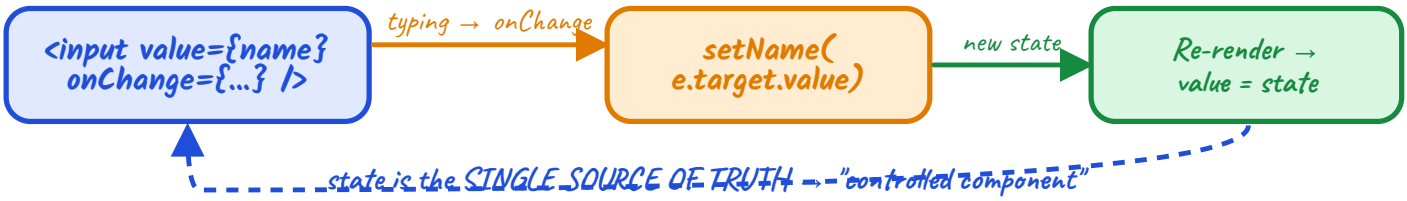
Remember

Handlers are plain functions; they can read state & call setters. Rendering must stay pure, but events are where side effects belong.

8. Forms & Controlled Inputs

Controlled component

An input whose **value is stored in React state** (**value + onChange**). React controls the input, so you can validate, format or reset it easily.



Controlled form – many fields with ONE state object

```
function SignUp() {
  const [form, setForm] = useState({ name: "", email: "", role: "student", agree: false });
  const [error, setError] = useState("");
  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setForm({ ...form, [name]: type === "checkbox" ? checked : value });   // [name] = computed key
  };
  const handleSubmit = (e) => {
    e.preventDefault();                                  // no page reload
    if (!form.email.includes("@")) return setError("Invalid email");
    console.log("Sending", form);  setError("");
  };
  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} placeholder="Name" />
      <input name="email" type="email" value={form.email} onChange={handleChange} />
      <select name="role" value={form.role} onChange={handleChange}>
        <option value="student">Student</option><option value="dev">Developer</option>
      </select>
      <label><input type="checkbox" name="agree" checked={form.agree} onChange={handleChange} /> I
agree</label>
      {error && <p style={{ color: "red" }}>{error}</p>}
      <button disabled={!form.agree}>Register</button>
    </form>);
}
```

Input type	Bind with
text / email / password / number / textarea / select	**value={x} onChange={e => setX(e.target.value)}**
checkbox	**checked={x} onChange={e => setX(e.target.checked)}**
radio	**checked={choice === "a"} onChange={() => setChoice("a")}**
file	**onChange={e => setFile(e.target.files[0])} (cannot set value)**

Controlled vs Uncontrolled

Uncontrolled (useRef)

```
const ref = useRef();
<input ref={ref} defaultValue="Hi" />
<button onClick={() => alert(ref.current.value)}>
```

- → **Controlled**: React state = truth → live validation, instant feedback (preferred).
- → **Uncontrolled**: DOM keeps the value; read it with a **ref** / **FormData**. Simpler for tiny forms.
- → Big forms: **React Hook Form** + **Zod** validation.

9. Conditional Rendering

Show different UI depending on **state / props**. It is plain JavaScript – use **if**, **? :** and **&&**.

1. if / early return

```
function Greeting({ isLoggedIn }) {
  if (!isLoggedIn) return <h2>Please log in</h2>;        // early return
  return <h2>Welcome back!</h2>;
}
```

2. Ternary operator ? : (either / or)

```
<button>{isOn ? "Turn OFF" : "Turn ON"}</button>
<p style={{ color: score >= 50 ? "green" : "red" }}>{score >= 50 ? "Pass" : "Fail"}</p>
```

3. Logical AND && (show or nothing)

```
{cart.length > 0 && <p>You have {cart.length} items</p>}
{isAdmin && <AdminPanel />}
```

&& trap with 0

{"{count &&

Items

}"} prints **0** when count is 0! Use **{"{count > 0 && ...}"}** or a ternary.

4. Variable / switch / object map

```
let content;
if (status === "loading") content = <Spinner />;
else if (status === "error") content = <p>Error!</p>;
else content = <List data={data} />;
return <div>{content}</div>;

const views = { home: <Home />, about: <About />, contact: <Contact /> };   // lookup object
return views[page] ?? <NotFound />;
```

Technique	Use when
if / early return	Whole component changes (loading, error, guard)
cond ? A : B	Choose between two pieces of JSX inside markup
cond && A	Show something or nothing
switch / object map	Many options (tabs, pages, status)
return null	Render nothing from a component

Real example – loading / error / data pattern & toggle

```
function Users() {
  const [show, setShow] = useState(true);
  const { data, loading, error } = useUsers();              // custom hook (Chapter 16)
  if (loading) return <p>⏳ Loading...</p>;
  if (error) return <p>❌ {error.message}</p>;
  return (<>
    <button onClick={() => setShow(!show)}>{show ? "Hide" : "Show"}</button>
    {show && <ul>{data.map(u => <li key={u.id}>{u.name}</li>)}</ul>}
  </>);
}
```

Also: **className={"{isActive ? 'btn active' : 'btn'}"}** for conditional CSS · **??** default value · **optional chaining** **user?.address?.city**. Components that return **null** / **false** render nothing.

10. Rendering Lists & Keys

Use the JavaScript array method **.map()** to turn data into JSX. Every item needs a unique **key**.

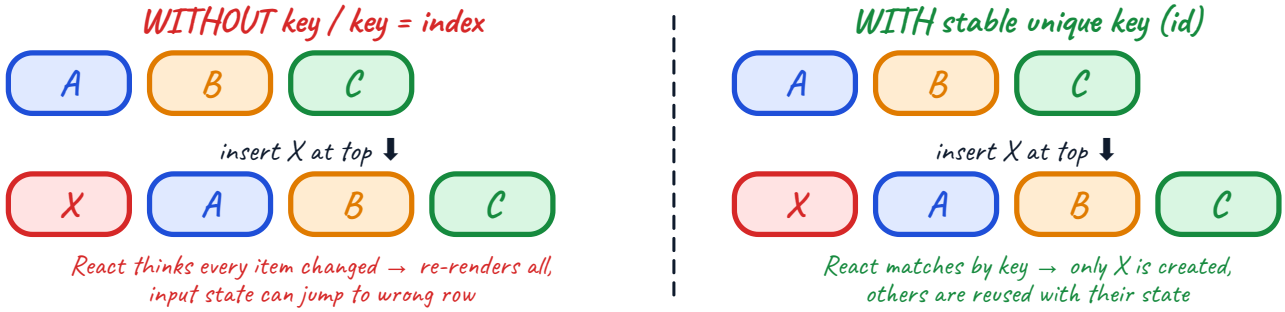
Basic list

```
const fruits = ["Apple", "Mango", "Banana"];
<ul>{fruits.map((f, i) => <li key={i}>{f}</li>)}</ul>          // ok only for static lists
```

List of objects → components (use id as key!)

```
const products = [
  { id: 1, name: "Laptop", price: 999, stock: true },
  { id: 2, name: "Phone",  price: 599, stock: false },
];
function ProductList() {
  return (
    <div className="grid">
      {products.map(p => (
        <ProductCard key={p.id} {...p} />          // key goes on the element INSIDE map
      ))}
    </div>
  );
}
function ProductCard({ name, price, stock }) { return <div><h3>{name}</h3><p>${price}</p></div>; }
```

Why keys?



- A **key** tells React **which item is which** between renders, so it can add / remove / reorder correctly and keep component state. Must be **unique among siblings** and **stable** (id from database).
- **Don't** use **Math.random()** (new key every render) and avoid index when the list can be sorted, filtered or changed. Keys are **not passed as a prop** (can't read **props.key**).

Filter, sort & search before rendering

```
const [search, setSearch] = useState("");
const visible = products
  .filter(p => p.name.toLowerCase().includes(search.toLowerCase()))   // search box
  .sort((a, b) => a.price - b.price);                                 // sort a COPY if state: [...arr].sort()
<input value={search} onChange={e => setSearch(e.target.value)} placeholder="Search..." />
{visible.length === 0 ? <p>No results</p> : visible.map(p => <ProductCard key={p.id} {...p} />)}
```

Task	Array method
Render	**map()**
Remove / filter	**filter()**
Total / sum	**reduce((s, p) => s + p.price, 0)**
Update one item	**map(i => i.id === id ? {"{...i, x}"} : i)**

Several elements per item without extra div: **<Fragment key={"{id}"}>...</Fragment>** (short **<>** cannot take a key).

11. useEffect & Component Lifecycle

Side effect

Anything that reaches **outside** rendering: fetching data, timers, event listeners, changing **document.title**, localStorage, subscriptions. **useEffect** runs this code **after React has updated the screen.**

Mount (1st render) → Effect runs after paint → Dependency changes → Cleanup of old effect runs → Effect runs again → Unmount → final cleanup

useEffect(() => { ...setup... ; return () => { ...cleanup... } }, [dependencies])

[] = run once · [a,b] = run when a or b change · no array = run after EVERY render

Syntax	When it runs
useEffect(() => {...})	After **every** render (rarely wanted)
useEffect(() => {...}, [])	Only **once** after first render (mount) – like componentDidMount
useEffect(() => {...}, [id, q])	On mount + whenever **id** or **q** changes
return () => {...} inside	**Cleanup** – before re-running & on unmount (componentWillUnmount)

1. Fetch data on mount

```
function Users() {
  const [users, setUsers] = useState([]); const [loading, setLoading] = useState(true);
  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(r => r.json()).then(d => { setUsers(d); setLoading(false); })
      .catch(console.error);
  }, []);                                          // [] → run once
  return loading ? <p>Loading...</p> : <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

2. Timer + cleanup

```
useEffect(() => {
  const id = setInterval(
    () => setSec(s => s + 1), 1000);
  return () => clearInterval(id); // stop
}, []);
```

3. Title & event listener

```
useEffect(() => {
  document.title = `Count: ${n}`;
}, [n]);                          // when n changes
useEffect(() => {
  const h = () => setW(innerWidth);
  addEventListener("resize", h);
  return () => removeEventListener("resize", h);
}, []);
```

Pitfalls

- **Missing dependencies** → stale values (ESLint rule exhaustive-deps).
- Setting state inside an effect **without deps** → infinite loop.
- **StrictMode** runs effects twice in dev – cleanup must undo setup.
- Can't make the effect callback **async**; define an async function inside.

You might not need an effect

Derive values during render (**const total = a + b**) and handle user actions in **event handlers**. Use effects only to **sync with outside systems.**

12. useRef – Refs & the DOM

useRef(initial) returns an object {"{ current: initial }"} that **stays the same between renders**. Changing **ref.current** does **not** cause a re-render.

useState
changes → RE-RENDER
value shown on screen
setState() to change

useRef
changes → NO re-render
ref.current persists
DOM element / timer id / previous value

Use 1 – access a DOM element (focus, scroll, measure)

```
function SearchBox() {
  const inputRef = useRef(null);
  useEffect(() => { inputRef.current.focus(); }, []);        // focus on mount
  return (<>
    <input ref={inputRef} placeholder="Search..." />
    <button onClick={() => inputRef.current.value = ""}>Clear</button>
  </>);
}
```

Use 2 – keep a mutable value

```
const timer = useRef(null);
const start = () => {
  timer.current = setInterval(tick, 1000);
};
const stop = () => clearInterval(timer.current);
```

Use 3 – previous value / render count

```
const prev = useRef();
useEffect(() => { prev.current = value; });
// prev.current = value of last render
const renders = useRef(0); renders.current++;
```

	useState	useRef
Triggers re-render	Yes	No
Persists across renders	Yes	Yes
Read/Write	setState (async snapshot)	ref.current (immediate, mutable)
Use for	Data shown in UI	DOM nodes, timers, previous values, uncontrolled inputs

Other related hooks

- **forwardRef / ref as prop (React 19)** – pass a ref to a child's inner DOM element: **<MyInput ref={"{r}"} />**.
- **useImperativeHandle** – expose only chosen methods (e.g. **focus()**) to the parent.
- **useLayoutEffect** – like useEffect but runs **before paint** (measure layout to avoid flicker).
- **useId** – unique id for labels/inputs: **const id = useId();** → **<label htmlFor={"{id}"}>**.

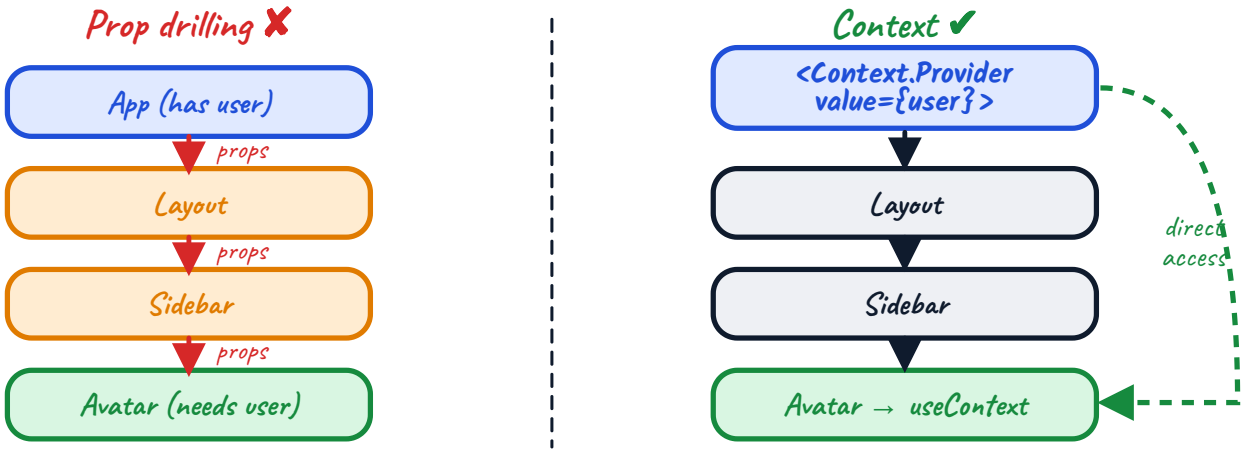
Rule

Don't read / write **ref.current during rendering** (except to initialise). Use refs inside event handlers and effects. Avoid manipulating the DOM directly when state can do it.

13. Context API & useContext

Context

A way to share data (theme, logged-in user, language, cart) with **many components without passing props at every level** – solves **prop drilling**.



3 steps: create → provide → consume

context/ThemeContext.jsx

```jsx
import { createContext, useContext, useState } from "react";
const ThemeContext = createContext(null);                    // 1. create

export function ThemeProvider({ children }) {                // 2. provider component
  const [theme, setTheme] = useState("light");
  const toggle = () => setTheme(t => (t === "light" ? "dark" : "light"));
  return (
    <ThemeContext.Provider value={{ theme, toggle }}>
      {children}
    </ThemeContext.Provider>
  );
}
export const useTheme = () => useContext(ThemeContext);      // 3. custom hook to consume
```

Use it anywhere

```jsx
// main.jsx →  <ThemeProvider><App /></ThemeProvider>
function Navbar() {
  const { theme, toggle } = useTheme();                      // no props needed!
  return <nav className={theme}><button onClick={toggle}>Switch to {theme === "light" ? "dark" : "light"}
</button></nav>;
}
```

Good uses

Theme / dark mode · authenticated user · language (i18n) · shopping cart · toast notifications · small global state.

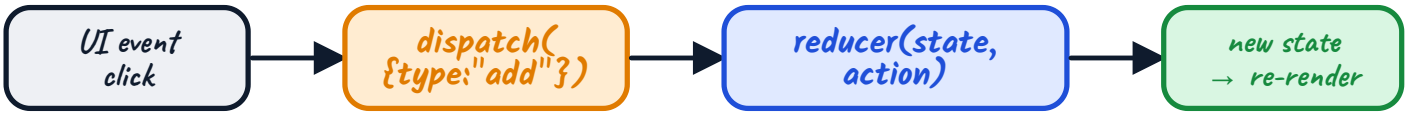
Limits

When **value** changes, **all** consumers re-render. Don't put fast-changing data (mouse position) in one big context; split contexts or use **Zustand / Redux** for large apps. Memoise the value: ***useMemo(() => ({"{theme}"}), [theme])***.

Default value (***createContext("light")***) is used only when there's **no Provider** above. Providers can be **nested** – the nearest one wins. React 19: ***<ThemeContext value={"{...}"}>*** can be used directly as provider.

14. useReducer

An alternative to **useState** for **complex state** (many related values or many ways to update). All update logic lives in **one reducer function**.



(state, action) ⟶ newState • pure function • no mutation • like a bank teller: actions in, new balance out

Counter / Todo with useReducer

```
const initialState = { todos: [], filter: "all" };

function reducer(state, action) {                    // pure: no mutation, no side effects
  switch (action.type) {
    case "add":    return { ...state, todos: [...state.todos, { id: Date.now(), text: action.text, done: false }] };
    case "toggle": return { ...state, todos: state.todos.map(t => t.id === action.id ? { ...t, done: !t.done } : t) };
    case "remove": return { ...state, todos: state.todos.filter(t => t.id !== action.id) };
    case "filter": return { ...state, filter: action.value };
    default: throw new Error("Unknown action " + action.type);
  }
}

function TodoApp() {
  const [state, dispatch] = useReducer(reducer, initialState);   // [state, dispatch]
  return (<>
    <button onClick={() => dispatch({ type: "add", text: "Learn React" })}>Add</button>
    {state.todos.map(t => (
      <li key={t.id} onClick={() => dispatch({ type: "toggle", id: t.id })}>
        {t.done ? "✅" : "⬜"} {t.text}
        <button onClick={() => dispatch({ type: "remove", id: t.id })}>x</button>
      </li>))}
  </>);
}
```

	useState	useReducer
Best for	Simple, independent values	Complex objects, many actions, next state depends on previous
Update	setX(value)	dispatch({"{type, payload}"})
Logic	Spread around handlers	Centralised in reducer → easy to test

Power combo

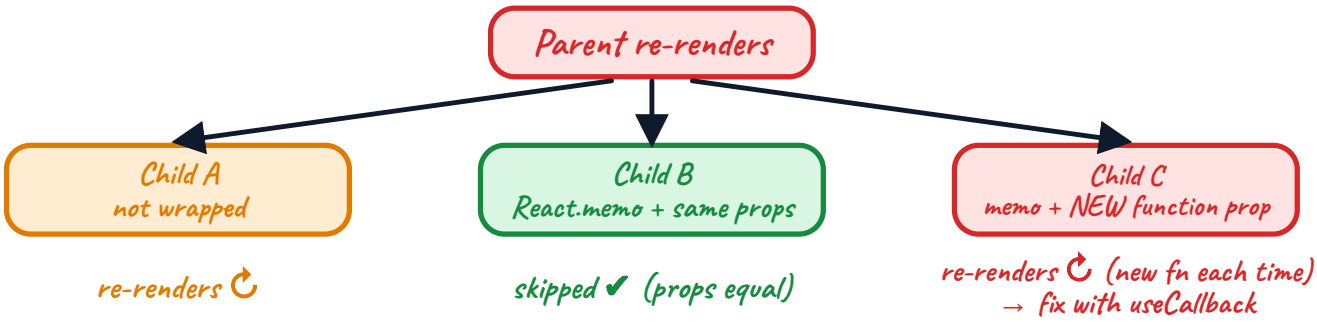
useReducer + **useContext** = mini Redux. Put **state** and **dispatch** in two contexts: **<StateCtx.Provider value={"{state}"}><DispatchCtx.Provider value={"{dispatch}"}>** - dispatch never changes, so its consumers don't re-render.

Remember

Reducer must be **pure** (same input → same output) and must **return a new object**. Lazy init: **useReducer(reducer, arg, initFn)**.

15. Performance: memo, useMemo, useCallback

A component re-renders when its **state changes**, its **parent re-renders**, or a **context** it uses changes. Usually that's fine – optimise only when something is **slow** (measure with the **React Profiler**).



Tool	What it caches	Use when
`React.memo(Comp)`	The **rendered result** of a component – skipped if props are shallow-equal	Heavy child that re-renders with same props
`useMemo(fn, deps)`	A **computed value**	Expensive calculation / filter of big lists; stable object/array props
`useCallback(fn, deps)`	A **function reference**	Passing callbacks to memoised children or as effect dependencies

React.memo + useCallback working together

```
const Row = React.memo(function Row({ item, onDelete }) {      // skip re-render if props same
  console.log("render", item.id);
  return <li>{item.name} <button onClick={() => onDelete(item.id)}>x</button></li>;
});

function List() {
  const [items, setItems] = useState(data);
  const [text, setText] = useState("");                          // typing here re-renders List...
  const handleDelete = useCallback(                              // ...but handleDelete keeps the same reference
    (id) => setItems(prev => prev.filter(i => i.id !== id)), []);
  return (<>
    <input value={text} onChange={e => setText(e.target.value)} />
    {items.map(i => <Row key={i.id} item={i} onDelete={handleDelete} />)}
  </>);
}
```

useMemo – skip heavy work

```
const filtered = useMemo(() => bigList.filter(x => x.name.includes(query)), [bigList, query]);
const total = useMemo(() => cart.reduce((s, i) => s + i.price * i.qty, 0), [cart]);
```

Other speed-ups

- → **Lazy load** routes: **`React.lazy`** + Suspense
- → **Virtualise** long lists (react-window)
- → **Debounce** search input
- → **useTransition / useDeferredValue** keep typing smooth
- → Keep state **close** to where it's used

Don't overuse

memo hooks cost memory & code. Don't wrap everything. Stable **keys**, small components and moving state down usually fix more. **React Compiler** (React 19) can auto-memoise.

16. Custom Hooks & All Hooks at a Glance

Custom hook

A JavaScript function whose **name starts with "use"** and that calls other hooks. It lets you **reuse stateful logic** between components (each call gets its own separate state).

hooks/useFetch.js

```
import { useState, useEffect } from "react";
export function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  useEffect(() => {
    const ctrl = new AbortController();                    // cancel on unmount
    setLoading(true);
    fetch(url, { signal: ctrl.signal })
      .then(r => { if (!r.ok) throw new Error(r.status); return r.json(); })
      .then(setData).catch(e => e.name !== "AbortError" && setError(e))
      .finally(() => setLoading(false));
    return () => ctrl.abort();
  }, [url]);
  return { data, loading, error };
}
// usage:  const { data, loading, error } = useFetch("/api/users");
```

More custom hook ideas: ***useLocalStorage(key, init)*** (state synced to localStorage) · ***useDebounce(value, ms)*** · ***useToggle*** · ***useWindowSize*** · ***useOnClickOutside*** · ***useAuth*** (wraps useContext).

All built-in hooks (cheat table)

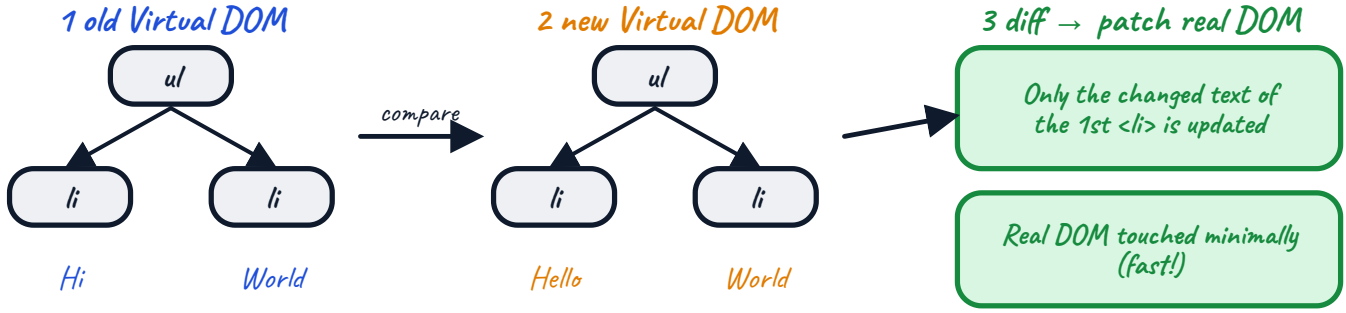
Hook	Purpose
useState	Local state
useReducer	Complex state with actions
useEffect / useLayoutEffect	Side effects after paint / before paint
useRef	DOM access, mutable value without re-render
useContext	Read context value
useMemo / useCallback	Cache value / function
useId	Unique ids
useTransition / useDeferredValue	Mark updates as low priority (smooth UI)
useSyncExternalStore	Subscribe to external stores (library authors)
React 19: use, useActionState, useFormStatus, useOptimistic	Read promises/context, form actions, pending state, instant UI

Rules of Hooks

1) Call hooks only at the **top level** – never inside if / loops / nested functions. 2) Call them only from **React function components or custom hooks.**

17. How React Renders – Virtual DOM & Reconciliation

- The **real DOM** is slow to update. React keeps a lightweight **Virtual DOM** (JS objects describing the UI). On every state change it builds a **new** virtual tree, **compares (diffs)** it with the old one and updates **only what changed** in the real DOM. This process is called **Reconciliation**.



The 3 phases: Trigger → Render → Commit

Phase	What happens
1. Trigger	Initial ***root.render()*** or a **state update** (setState / dispatch / context change).
2. Render	React **calls your components** to compute the new JSX (virtual DOM). Must be **pure**. No DOM changes yet.
3. Commit	React applies the minimal changes to the real DOM → browser **paints** → then runs ***useLayoutEffect*** / ***useEffect***.

Diffing rules (heuristics)

- **Different element type** (** ** → ****, or Card → Panel): old subtree is **destroyed**, new one built (state lost).
- **Same type**: DOM node is kept; only changed **attributes / text** are updated, then children are compared.
- **Lists**: matched by **key** (see Chapter 10). A **changed key** = new component instance – a trick to **reset state**: ***<Form key={"{userId}"} />***.

When does a component re-render?

- Its own **state** changes
- Its **parent** re-renders (even if props are the same!)
- A **context** it reads changes
- **Not** when a ref or normal variable changes
- **Same value** in setState (***Object.is*** equal) → bail out
- **Batching**: many setStates in one event = one render (React 18)

Snapshot behaviour – a classic interview question

```
function Demo() {
  const [n, setN] = useState(0);
  function click() {
    setN(n + 1); setN(n + 1); setN(n + 1);          // n is 0 in this render → ends as 1, not 3
    setN(p => p + 1); setN(p => p + 1);             // updater functions queue up → +2
    console.log(n);                                 // still the OLD value (snapshot)
  }
  return <button onClick={click}>{n}</button>;
}
```

18. React Router – Navigation

React has no built-in routing. **React Router** maps **URLs** → **components** in a SPA without page reloads. Install: ***npm i react-router-dom*** (v6/v7).



<Layout> (Navbar + <Outlet/> + Footer) → child route renders inside <Outlet/> · <Link> changes URL without reload

main.jsx + App.jsx – basic setup

```jsx
import { BrowserRouter, Routes, Route, Link, NavLink, Outlet, useParams, useNavigate } from "react-router-dom";

<BrowserRouter><App /></BrowserRouter>                    // in main.jsx

function App() {
  return (
    <Routes>
      <Route path="/" element={<Layout />}>                  {/* parent layout route */}
        <Route index element={<Home />} />                    {/* "/"            */}
        <Route path="about" element={<About />} />            {/* "/about"       */}
        <Route path="products/:id" element={<Product />} />   {/* dynamic param */}
        <Route path="dashboard/*" element={<Dashboard />} />   {/* nested wildcard */}
        <Route path="*" element={<NotFound />} />              {/* 404 */}
      </Route>
    </Routes>);
}
function Layout() {
  return (<>
    <nav><NavLink to="/" end>Home</NavLink> <NavLink to="/about">About</NavLink></nav>   {/* NavLink adds "active" class */}
    <Outlet />                                          {/* child route renders here */}
    <footer>© asadokz18</footer></>);
}
```

Hooks you will use

```jsx
const { id } = useParams();        // /products/42 → "42"
const navigate = useNavigate();
navigate("/login");  navigate(-1);   // go back
const [sp, setSp] = useSearchParams();
sp.get("q");  // /search?q=react
const loc = useLocation();         // pathname, state
```

Component	Use
<Link to>	Navigate w/o reload (never plain <a>)
<NavLink>	Link with active style
<Navigate to />	Redirect
<Outlet />	Nested route slot

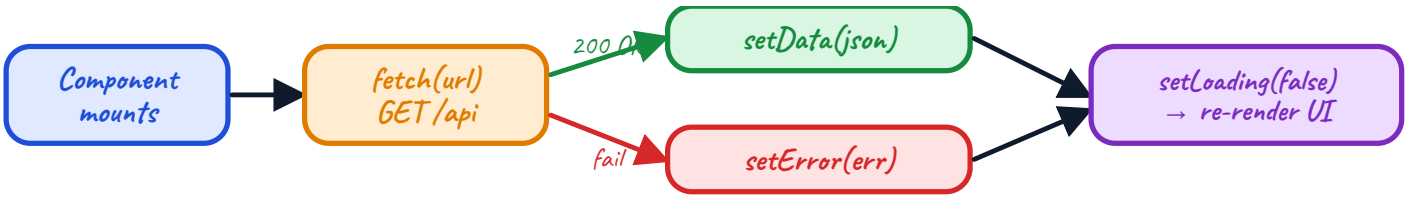
Protected route (authentication guard)

```jsx
function ProtectedRoute({ children }) {
  const { user } = useAuth();
  return user ? children : <Navigate to="/login" replace />;
}
<Route path="/dashboard" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
```

Router types: ***BrowserRouter*** (clean URLs, needs server fallback to index.html) · ***HashRouter*** (***/#/about***) · ***createBrowserRouter + RouterProvider*** (data router: **loader**, **action**, errorElement) – recommended in new apps. Lazy pages: ***const About = lazy(() => import("./About"))***.

19. Data Fetching & APIs

React doesn't fetch data itself - you call an API with ***fetch*** or ***axios*** inside ***useEffect*** (or a data library) and store the result in **state**.



Always handle 3 states in the UI: Loading... · Error · Data

fetch with async/await (full pattern)

```
function Posts() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  useEffect(() => {
    async function load() {                          // effect callback itself can't be async
      try {
        const res = await fetch("https://jsonplaceholder.typicode.com/posts?_limit=5");
        if (!res.ok) throw new Error("HTTP " + res.status);   // fetch doesn't reject on 404/500!
        setPosts(await res.json());
      } catch (e) { setError(e.message); }
      finally { setLoading(false); }
    }
    load();
  }, []);
  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}
```

POST with axios

```
import axios from "axios";
const api = axios.create({ baseURL: "/api" });
await api.post("/todos", { text: "Learn" });
await api.put(`/todos/${id}`, data);
await api.delete(`/todos/${id}`);
```

Method	Purpose
GET	read
POST	create
PUT / PATCH	replace / update
DELETE	remove

Better: TanStack Query (React Query)

```
const { data, isLoading, error } = useQuery({ queryKey: ["posts"], queryFn: () => fetch("/api/posts").then(r => r.json()) });
const mutation = useMutation({ mutationFn: newPost => axios.post("/api/posts", newPost),
  onSuccess: () => queryClient.invalidateQueries({ queryKey: ["posts"] }) });      // refetch after change
```

- Gives **caching, auto-refetch, retries, pagination, loading/error states** - far less code than useEffect. Alternatives: **SWR**, RTK Query.
- **Avoid race conditions**: cleanup with ***AbortController*** or an ***ignore*** flag. Store **API URL / keys** in **.env** (***VITE_API_URL***; Vite exposes only ***VITE_*** vars via ***import.meta.env***) - never put secrets in front-end code.

20. Styling in React

Method	Example	Notes
1. Plain CSS file	`import "./App.css"; <div className="card">`	Global - class names can clash
2. Inline style	`style={{ color:'red', fontSize:20 }}`	Object, camelCase; no :hover / media queries
3. CSS Modules	`import s from "./Btn.module.css"; className={s.btn}`	**Scoped** automatically (hashed names)
4. Tailwind CSS	`className="bg-blue-500 p-4 rounded-lg hover:bg-blue-600"`	Utility classes, very popular
5. CSS-in-JS	`styled-components / Emotion`	Styles in JS, dynamic props
6. UI libraries	`MUI, Chakra UI, shadcn/ui, Bootstrap`	Ready-made components

CSS Modules

```
/* Button.module.css */
.btn { background: royalblue; color: #fff; }
.danger { background: crimson; }
// Button.jsx
import s from "./Button.module.css";
<button className={`${s.btn} ${bad ? s.danger : ""}`}>OK</button>
```

Tailwind (setup: npm i tailwindcss @tailwindcss/vite)

```
  return (
    <div className="max-w-sm rounded-xl shadow-md p-6 bg-white">
      <h2 className="text-xl font-bold text-gray-800">
        {title}</h2>
      <button className="mt-4 px-4 py-2 bg-indigo-600 text-white rounded hover:bg-indigo-700">Buy</button>
    </div>);
}
```

styled-components

```
import styled from "styled-components";
const Button = styled.button`
  background: ${props => (props.primary ? "royalblue" : "white")};
  color: ${props => (props.primary ? "white" : "royalblue")};
  padding: 8px 16px;  &:hover { opacity: .8; }
`;
<Button primary>Save</Button>
```

Conditional / dynamic classes

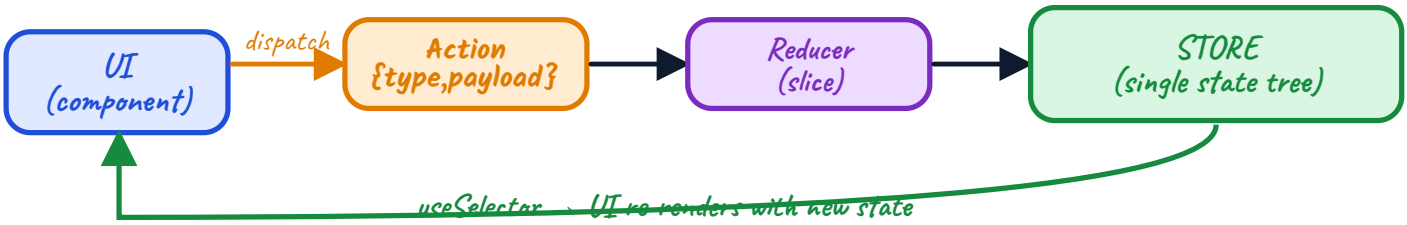
```
<li className={done ? "item done" : "item"}>                    // ternary
<li className={`item ${active ? "active" : ""}`}>               // template literal
import clsx from "clsx";    <li className={clsx("item", { done, active })}>      // clsx / classnames helper
<div style={{ width: `${percent}%` }} />                        // dynamic inline value
```

Which to choose?

Learning: plain CSS / CSS Modules. Real projects: **Tailwind** (+ shadcn/ui) or CSS Modules. Use **CSS variables** for dark mode: ***document.documentElement.dataset.theme = theme***. Icons: ***react-icons***, ***lucide-react***. Animation: **Framer Motion**.

21. State Management – Redux Toolkit & Zustand

As apps grow, many components need the **same global data** (user, cart, notifications). Options: **lift state** → **Context** → **Redux Toolkit / Zustand**. Server data → **React Query**.



one-way flow: UI → action → reducer → store → UI

Redux Toolkit (RTK) – the modern Redux

store/cartSlice.js + store.js

```
import { createSlice, configureStore } from "@reduxjs/toolkit";
const cartSlice = createSlice({
  name: "cart", initialState: { items: [] },
  reducers: {                                          // Immer lets you "mutate" safely here
    add:    (state, action) => { state.items.push(action.payload); },
    remove: (state, action) => { state.items = state.items.filter(i => i.id !== action.payload); },
  },
});
export const { add, remove } = cartSlice.actions;
export const store = configureStore({ reducer: { cart: cartSlice.reducer } });
```

Use in components (main.jsx: <Provider store={store}><App/></Provider>)

```
import { useSelector, useDispatch } from "react-redux";
function Cart() {
  const items = useSelector(state => state.cart.items);        // read (re-renders when it changes)
  const dispatch = useDispatch();
  return (<>
    <h3>Cart ({items.length})</h3>
    <button onClick={() => dispatch(add({ id: 1, name: "Laptop" }))}>Add</button>
    <button onClick={() => dispatch(remove(1))}>Remove</button>
  </>);
}
```

Zustand – tiny & simple (npm i zustand)

```
import { create } from "zustand";
const useCart = create((set) => ({
  items: [],
  add: (item) => set((s) => ({ items: [...s.items, item] })),
  clear: () => set({ items: [] }),
}));
const items = useCart((s) => s.items);       // no Provider needed!
```

Tool	Pros	Use when
Context	Built-in, no library	Theme, auth, small shared state
Redux Toolkit	Predictable, DevTools, middleware	Large apps, big teams
Zustand	Minimal, fast, no boilerplate	Medium apps, simple global store

22. Patterns & Best Practices

Common component patterns

Pattern	Idea & example
Container / Presentational	Container fetches & holds state; presentational component only shows UI from props. (Hooks make this easier.)
Composition (children)	**`<Modal><LoginForm /></Modal>` – prefer composition over inheritance**
Higher-Order Component	Function that takes a component, returns a new one: **`const Safe = withAuth(Dashboard)`** (old style)
Render props	Prop that is a function returning JSX: **`<Mouse render={"{pos =>`** **`{pos.x}`** **`}"} />`**
Compound components	Components that work together: **`<Tabs><Tabs.List/><Tabs.Panel/></Tabs>`** (share state via context)
Custom hooks	Extract reusable logic - the modern replacement for HOC / render props
Controlled vs uncontrolled	Parent owns the value (props + onChange) or the component owns it

Compound component example

```
const TabsCtx = createContext();
function Tabs({ children, defaultTab }) {
  const [active, setActive] = useState(defaultTab);
  return <TabsCtx.Provider value={{ active, setActive }}>{children}</TabsCtx.Provider>;
}
Tabs.Tab = ({ id, children }) => { const { active, setActive } = useContext(TabsCtx);
  return <button className={active === id ? "on" : ""} onClick={() => setActive(id)}>{children}</button>; };
Tabs.Panel = ({ id, children }) => useContext(TabsCtx).active === id ? <div>{children}</div> : null;
```

Project structure & best practices

feature-based folders

```
src/
├─ components/   Button.jsx Modal.jsx   # shared UI
├─ features/
│   ├─ auth/      LoginForm.jsx authSlice.js
│   └─ cart/      Cart.jsx useCart.js
├─ pages/        Home.jsx Product.jsx
├─ hooks/        useFetch.js
├─ context/ · services/api.js · utils/
└─ App.jsx  main.jsx
```

- Small, single-purpose components
- **Keys** on lists, **immutable** state updates
- Derive data instead of storing it twice (**`const total = ...`**)
- Keep state as **local** as possible; lift only when needed
- Handle **loading / error / empty** states
- **Accessibility**: semantic tags, **`alt`**, labels, keyboard, aria-*
- Use **TypeScript**, ESLint, Prettier
- Don't copy props into state; no business logic in JSX

Rules of Hooks (again)

Top level only · only in components / custom hooks · effects need correct dependency arrays · state is immutable · components are pure functions of props + state.

23. Advanced React

Code splitting: lazy + Suspense

```
const Dashboard = React.lazy(() => import("./Dashboard"));        // loaded only when needed (smaller first bundle)
<Suspense fallback={<p>Loading page...</p>}><Dashboard /></Suspense>
```

Error Boundaries – catch render errors

```
class ErrorBoundary extends React.Component {                    // only class components (or react-error-boundary lib)
  state = { hasError: false };
  static getDerivedStateFromError() { return { hasError: true }; }
  componentDidCatch(err, info) { console.error(err, info); }
  render() { return this.state.hasError ? <h2>Something went wrong 😢</h2> : this.props.children; }
}
<ErrorBoundary><Profile /></ErrorBoundary>          // does NOT catch errors in event handlers / async code
```

Portals – render outside parent DOM

```
import { createPortal } from "react-dom";
function Modal({ children }) {
  return createPortal(
    <div className="overlay">{children}</div>,
    document.body);        // modal, tooltip, toast
}
```

useTransition – keep UI responsive

```
const [isPending, startTransition] = useTransition();
function onChange(e) {
  setText(e.target.value);                  // urgent
  startTransition(() =>                     // low priority
    setResults(filterBig(e.target.value)));
}
```

What's new in React 19

- Actions: **<form action={"{fn}"}>** with **useActionState**, **useFormStatus**, **useOptimistic** (instant UI before server replies).
- **use(promise)** hook reads promises/context • **ref as prop** (no forwardRef) • **<title>, <meta>** tags inside components • **Server Components** & **Server Actions** • React Compiler (auto-memo).

Rendering strategies & Next.js

CSR (Vite SPA)	SSR (Next.js)	SSG
empty HTML → JS downloads → renders in browser	server renders HTML per request → hydrate in browser	HTML built at BUILD time → served from CDN
Fast after load weak SEO, slow 1st paint	Good SEO, fresh data server cost	Fastest, cheap static content

- Next.js is the React framework: file-based routing (**app/about/page.js**), SSR / SSG / ISR, API routes, image optimisation, **Server Components**. Alternatives: **Remix / React Router v7**, **Astro**.
- **React Native** uses React to build iOS/Android apps. Also: **Strict Mode**, **Fragments**, **Profiler**, **i18n** (react-i18next), **animations** (Framer Motion).

24. Testing & Deployment

Testing React components

→ **Vitest** (or Jest) = test runner · **React Testing Library** = test like a user (find by text / role, click, type) · **Playwright / Cypress** = end-to-end (E2E).

```
npm i -D vitest jsdom @testing-library/react @testing-library/jest-dom @testing-library/user-event
npm run test          # "test": "vitest" in package.json
```

Counter.test.jsx

```
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import Counter from "./Counter";

test("increments the count when the button is clicked", async () => {
  render(<Counter />);                                          // 1. Arrange
  expect(screen.getByText(/count: 0/i)).toBeInTheDocument();
  await userEvent.click(screen.getByRole("button", { name: "+" }));   // 2. Act
  expect(screen.getByText(/count: 1/i)).toBeInTheDocument();     // 3. Assert
});
```

Query / matcher	Meaning
getByText / getByRole / getByLabelText	Find element (throws if missing) · ***queryBy...*** returns null · ***findBy...*** waits (async)
toBeInTheDocument / toHaveTextContent	jest-dom assertions · ***vi.fn()*** mock functions · ***vi.mock()*** mock modules / API

Test **behaviour** (what the user sees), not implementation details. Pyramid: many unit tests → some integration → few E2E.

Build & deploy

```
npm run build       # creates dist/ (minified, tree-shaken, hashed files)
npm run preview     # test the production build locally
```

Platform	How
Vercel / Netlify	Connect GitHub repo → build command ***npm run build***, output ***dist*** → auto-deploy on every push (free tier, HTTPS, preview URLs)
GitHub Pages	Set ***base: "/repo-name/"*** in vite.config.js; deploy ***dist*** with GitHub Actions or gh-pages
Firebase / Cloudflare	***firebase deploy*** · Cloudflare Pages
Own server / Docker	Serve ***dist*** with Nginx; add fallback ***try_files $uri /index.html*** so React Router URLs work on refresh

Before you ship

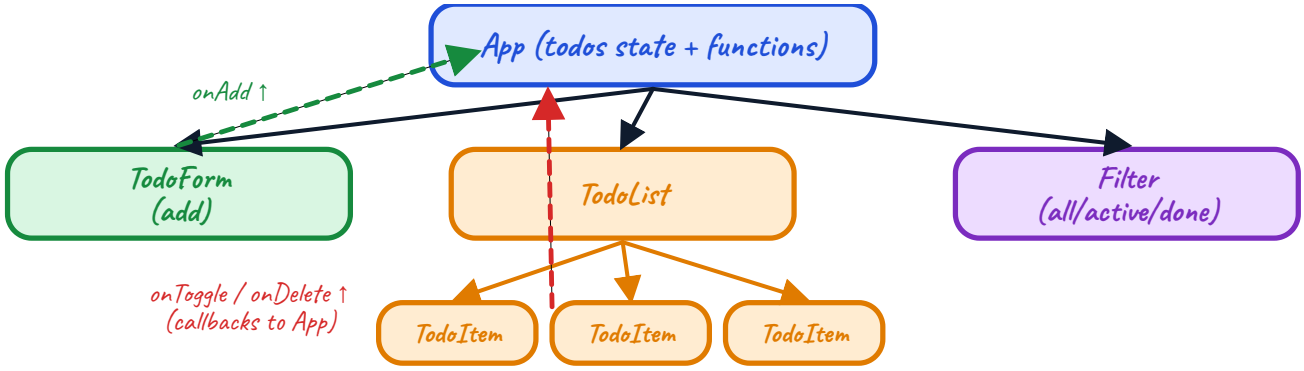
→ Env vars in host settings (***VITE_API_URL***)
→ Check Lighthouse (performance, a11y, SEO)
→ Remove console.logs; add error tracking (Sentry)

CI/CD

GitHub Actions: on push → ***npm ci*** → ***npm test*** → ***npm run build*** → deploy.

25. Mini Project – Todo App (uses everything!)

Features: add, toggle done, delete, filter (all / active / done), edit count, **saved in localStorage. Skills used:** components, props, useState, events, forms, lists & keys, useEffect, conditional rendering.



state lives in App ⬇ props down • ⬆ callbacks up

App.jsx

```
import { useState, useEffect } from "react";
import TodoForm from "./TodoForm"; import TodoList from "./TodoList"; import Filter from "./Filter";

export default function App() {
  const [todos, setTodos] = useState(() => JSON.parse(localStorage.getItem("todos")) ?? []);   // lazy init
  const [filter, setFilter] = useState("all");

  useEffect(() => { localStorage.setItem("todos", JSON.stringify(todos)); }, [todos]);        // persist

  const addTodo = (text) => setTodos([...todos, { id: Date.now(), text, done: false }]);
  const toggle  = (id) => setTodos(todos.map(t => t.id === id ? { ...t, done: !t.done } : t));
  const remove  = (id) => setTodos(todos.filter(t => t.id !== id));

  const visible = todos.filter(t => filter === "all" ? true : filter === "done" ? t.done : !t.done);
  const left = todos.filter(t => !t.done).length;

  return (
    <main className="app">
      <h1>📝 My Todos</h1>
      <TodoForm onAdd={addTodo} />
      <Filter value={filter} onChange={setFilter} />
      {visible.length === 0 ? <p>Nothing here 🎉</p> : <TodoList todos={visible} onToggle={toggle} onDelete=
{remove} />}
      <p>{left} item{left !== 1 && "s"} left</p>
    </main>
  );
}
```

25. Todo App – other components & styling

TodoForm.jsx

```
export default function TodoForm({ onAdd }) {
  const [text, setText] = useState("");
  const submit = (e) => {
    e.preventDefault();
    if (!text.trim()) return;
    onAdd(text.trim());  setText("");
  };
  return (
    <form onSubmit={submit}>
      <input value={text} placeholder="New task..."
        onChange={(e) => setText(e.target.value)}
/>
      <button>Add</button>
    </form>);
}
```

TodoList.jsx + Filter.jsx

```
export default function TodoList({ todos, onToggle,
onDelete }) {
  return <ul>{todos.map(t => (
    <li key={t.id} className={t.done ? "done" : ""}>
      <input type="checkbox" checked={t.done}
        onChange={() => onToggle(t.id)} />
      {t.text} <button onClick={() =>
onDelete(t.id)}>🗑</button>
    </li>))}</ul>;
}
export default function Filter({ value, onChange })
{
  return ["all","active","done"].map(f =>
    <button key={f} disabled={value === f}
      onClick={() => onChange(f)}>{f}</button>);
}
```

App.css – minimal styling

```
.app { max-width: 420px; margin: 40px auto; font-family: sans-serif; }
li { display: flex; gap: 8px; align-items: center; padding: 6px 0; }
li.done { text-decoration: line-through; color: gray; }
button:disabled { background: royalblue; color: #fff; }
```

Steps to build it yourself

1. Create the Vite app. 2. Make the 4 files above in **src/**. 3. Start with **static JSX**, then add **state**. 4. Pass **props & callbacks**. 5. Add filter + localStorage. 6. Style it. 7. Push to **GitHub** and deploy on **Vercel**.

Project ideas to practise

Weather app (API) · Quiz app · Expense tracker (useReducer + chart) · Movie search (TMDB API + Router) · E-commerce cart (Context / Redux) · Notes app with localStorage · Blog with React Router + Markdown · Chat UI · Dark-mode portfolio. **Upload to GitHub & deploy on Vercel.**

26. React Interview Questions & Answers

1. **What is React?** A JavaScript library (by Meta) for building UIs from reusable components using a declarative approach and a Virtual DOM.
2. **Library or framework?** A **library** for the view layer; routing, state, data fetching come from other libs (Next.js makes it a full framework).
3. **What is JSX?** HTML-like syntax in JavaScript, compiled to ***React.createElement***. Needs one root, camelCase attributes, ***className***.
4. **What is the Virtual DOM?** A lightweight JS copy of the UI. React diffs old vs new virtual trees and updates only changed real-DOM nodes (reconciliation).
5. **Functional vs class components?** Functions + Hooks are modern and shorter; classes use ***this***, ***setState*** and lifecycle methods.
6. **Props vs state?** Props: inputs from parent, read-only. State: internal, changeable data that triggers re-render.
7. **What is the key prop?** A unique, stable id on list items so React can track identity during diffing. Avoid index when the list changes.
8. **Controlled vs uncontrolled components?** Controlled: value held in state (***value + onChange***). Uncontrolled: DOM holds value, read with a ref.
9. **What is useState? Why not mutate state?** Hook for local state. React detects changes by new reference; mutation doesn't trigger re-render and breaks memoisation.
10. **Explain useEffect & dependency array.** Runs side effects after render. ***[]*** once · ***[x]*** when x changes · none every render. Return a cleanup function.
11. **useRef vs useState?** useRef holds a mutable value / DOM node; changing it does **not** re-render.
12. **What is prop drilling? How to avoid it?** Passing props through many levels. Use Context, composition (children) or a state library.
13. **useMemo vs useCallback vs React.memo?** Cache a value / a function / a component's render. Use to avoid expensive recalculation and needless re-renders.
14. **What is lifting state up?** Moving shared state to the nearest common parent and passing it down.
15. **Rules of Hooks?** Call only at top level (no loops/conditions), only in components or custom hooks.
16. **What is a custom hook?** A function starting with ***use*** that reuses stateful logic (useFetch, useLocalStorage).
17. **What is useReducer?** State with a reducer ***(state, action) => newState*** for complex logic; dispatch actions.
18. **What is Context API?** Shares data across the tree without props: createContext → Provider → useContext.
19. **Synthetic events?** React's cross-browser wrapper around native events (onClick etc.).
20. **What are Fragments?** ***<>...</>*** group elements without adding an extra DOM node.

26. Interview Q&A (continued)

1. **When does a component re-render?** State change, parent re-render, or consumed context change. Not for ref changes or same-value state.
2. **How does React batching work?** Multiple state updates in one event/async are grouped into a single re-render (automatic batching in React 18).
3. **Why does *setCount(count+1)* three times add only 1?** State is a snapshot per render; use ***setCount(c => c+1)***.
4. **What is reconciliation?** The diffing algorithm comparing virtual trees: same type = update, different type = replace, lists matched by key.
5. **What is React.lazy & Suspense?** Code splitting - load a component on demand and show a fallback while loading.
6. **What are Error Boundaries?** Class components catching render errors in children and showing fallback UI (not for event handlers or async).
7. **What are Portals?** ***createPortal*** renders children into another DOM node (modals, tooltips).
8. **What is a Higher-Order Component?** A function that takes a component and returns an enhanced one (***withAuth***). Hooks replaced most uses.
9. **What is StrictMode?** Dev-only wrapper that double-invokes renders & effects to find impure code / missing cleanup.
10. **Client-side vs server-side rendering?** CSR renders in the browser (SPA); SSR renders HTML on server per request (better SEO, first paint); SSG at build time. Hydration attaches JS.
11. **What are Server Components?** Components rendered on the server only (no client JS, direct DB access) - React 18+/Next.js.
12. **Redux flow?** UI dispatches action → reducer computes new state → store updates → subscribed components re-render. RTK simplifies it.
13. **Redux vs Context?** Context is for passing data; Redux adds predictable updates, middleware, DevTools, selectors - for large state.
14. **How to optimise a React app?** memo/useMemo/useCallback, lazy loading, list virtualisation, stable keys, split components, avoid inline heavy work, Profiler, production build.
15. **How do you handle forms/validation?** Controlled inputs + state, or React Hook Form + Zod/Yup.
16. **How to fetch data?** fetch/axios in useEffect, or TanStack Query for caching & states. Handle loading / error; cancel with AbortController.
17. **How does routing work?** React Router matches the URL to ***<Route>***, ***Link*** avoids reload, ***useParams*** reads params, ***Outlet*** renders nested routes.
18. **What is *forwardRef*?** Passes a ref to a child's DOM node (React 19: ref is a normal prop).
19. **Why can't components be defined inside components?** A new function (identity) each render → React remounts it and loses state.
20. **How to test React?** Vitest/Jest + React Testing Library (user-centric queries), E2E with Playwright/Cypress.
21. **What is useLayoutEffect?** Synchronous effect before paint; for measuring DOM to avoid flicker.
22. **What's new in React 19?** Actions, useActionState / useOptimistic / use(), ref as prop, metadata tags, Compiler, Server Components stable.

Interview tip

Always give a tiny example: "I keep the cart in a context, memoise the Row with React.memo, fetch with React Query and deploy on Vercel."

27. React Cheat Sheet

Setup

```
npm create vite@latest app -- --template react
cd app && npm i && npm run dev
npm i react-router-dom axios zustand
```

Component + props + state

```
function Card({ title, children }) {
  const [n, setN] = useState(0);
  return <div className="card">
    <h3>{title}</h3>{children}
    <button onClick={() => setN(p => p + 1)}>{n}
</button>
  </div>;
}
export default Card;
```

Render patterns

```
{ok ? <A /> : <B />}     {ok && <A />}
{list.map(x => <Li key={x.id} {...x} />)}
<input value={v} onChange={e =>
setV(e.target.value)} />
<form onSubmit={e => { e.preventDefault(); }}>
```

Update state immutably

```
setObj({ ...obj, k: 1 });  setArr([...arr, x]);
setArr(arr.filter(i => i.id !== id));
setArr(arr.map(i => i.id === id ? { ...i, a: 1 } : i));
```

Hooks

```
useState(0)              useReducer(reducer, init)
useEffect(() => {       // side effects
  ...; return () => cleanup;
}, [deps]);             // [] once
const ref = useRef(null);       // ref.current
const v = useMemo(() => calc(a), [a]);
const f = useCallback(() => go(a), [a]);
const ctx = useContext(MyCtx);
const id = useId();
```

Router

```
<Routes><Route path="/" element={<Home />} />
 <Route path="p/:id" element={<P />} />
 <Route path="*" element={<NotFound />} />
</Routes>
<Link to="/about">About</Link>
const { id } = useParams();  const go =
useNavigate();
```

Context

```
const Ctx = createContext(null);
<Ctx.Provider value={{ user }}>...</Ctx.Provider>
const { user } = useContext(Ctx);
```

Performance

```
const C = React.memo(Child);
const Page = lazy(() => import("./Page"));
<Suspense fallback={<p>...</p>}><Page />
</Suspense>
```

Golden rules

Components = pure functions · props down, events up · never mutate state · keys on lists · hooks at top level · effects only for outside-world sync · start simple, optimise later.

⚛ Keep building projects – that is how React becomes easy! ⚛ — asadokz18